In [3]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.metrics import precision_score, recall_score, f1_score

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
```

In [4]:
```
df = pd.read_csv("smartphone_overheating_pbl_final.csv")
df.head()
```

Out[4]:

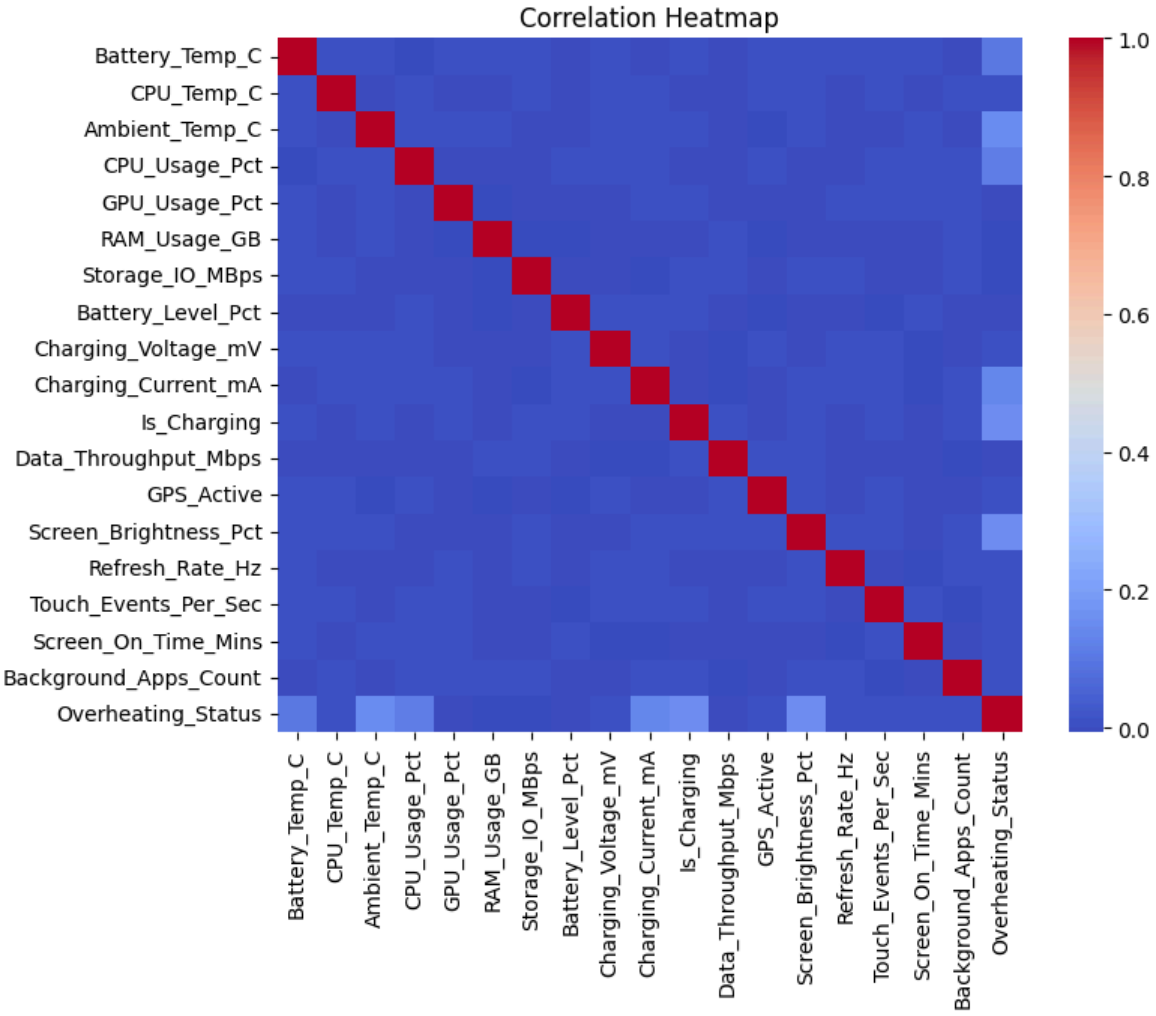
	Battery_Temp_C	CPU_Temp_C	Ambient_Temp_C	CPU_Usage_Pct	GPU_Usage_
0	38.483571	50.244757	29.510183	35.980335	0.235
1	35.308678	32.757162	38.118758	63.838575	55.485
2	39.238443	46.603498	44.661971	86.824548	44.509
3	43.615149	37.046092	43.407392	33.084809	93.555
4	34.829233	39.380778	25.699396	14.717118	4.915

In [5]:
```
print(df.shape)
print(df.info())
```

```
(100000, 19)
<class 'pandas.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 19 columns):
 #   Column                 Non-Null Count   Dtype
---  ------                 --------------   -----
 0   Battery_Temp_C         100000 non-null  float64
 1   CPU_Temp_C             100000 non-null  float64
 2   Ambient_Temp_C         100000 non-null  float64
 3   CPU_Usage_Pct          100000 non-null  float64
 4   GPU_Usage_Pct          100000 non-null  float64
 5   RAM_Usage_GB           100000 non-null  float64
 6   Storage_IO_MBps        100000 non-null  float64
 7   Battery_Level_Pct      100000 non-null  float64
 8   Charging_Voltage_mV    100000 non-null  float64
 9   Charging_Current_mA    100000 non-null  float64
 10  Is_Charging            100000 non-null  int64
 11  Data_Throughput_Mbps   100000 non-null  float64
 12  GPS_Active             100000 non-null  int64
 13  Screen_Brightness_Pct  100000 non-null  int64
 14  Refresh_Rate_Hz        100000 non-null  int64
 15  Touch_Events_Per_Sec   100000 non-null  float64
 16  Screen_On_Time_Mins    100000 non-null  float64
 17  Background_Apps_Count  100000 non-null  int64
 18  Overheating_Status     100000 non-null  int64
dtypes: float64(13), int64(6)
memory usage: 14.5 MB
None
```

In [7]:

```
plt.figure(figsize=(8,6))

corr = df.corr()

sns.heatmap(corr, cmap="coolwarm")

plt.title("Correlation Heatmap")
plt.show()
```



In [8]:
```
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
```

In [9]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [10]:
```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [11]:
```
lr = LogisticRegression(max_iter=500)

dt = DecisionTreeClassifier(
    max_depth=5,
    min_samples_split=10
)

rf = RandomForestClassifier(n_estimators=50)

knn = KNeighborsClassifier(n_neighbors=5)

svm = SVC()
```

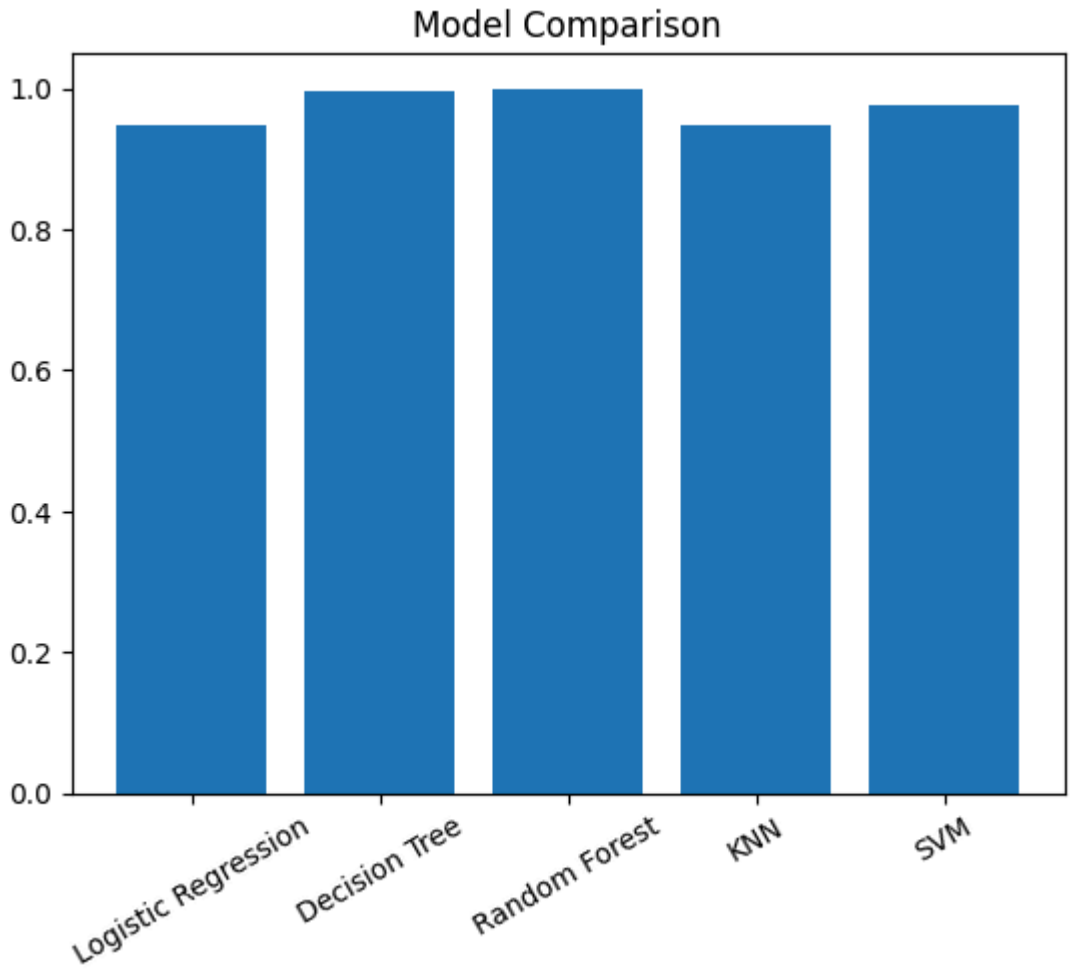
In [14]:
```
models = {
    "Logistic Regression": lr,
```

```
    "Decision Tree": dt,
    "Random Forest": rf,
    "KNN": knn,
    "SVM": svm
}

results = {}

for name, model in models.items():
    print(f"Running {name}...")

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    results[name] = acc

    print(name, "Accuracy:", acc)
```

```
Running Logistic Regression...
Logistic Regression Accuracy: 0.9474
Running Decision Tree...
Decision Tree Accuracy: 0.9974
Running Random Forest...
Random Forest Accuracy: 1.0
Running KNN...
KNN Accuracy: 0.94895
Running SVM...
SVM Accuracy: 0.9764
```

In [16]:

```
plt.bar(results.keys(), results.values())
plt.xticks(rotation=30)
plt.title("Model Comparison")
plt.show()
```



In [17]:
```
depths = [3, 5, 7]
best_acc = 0
best_depth = 0

for d in depths:
    model = DecisionTreeClassifier(max_depth=d)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    print("Depth:", d, "Accuracy:", acc)

    if acc > best_acc:
        best_acc = acc
        best_depth = d

print("Best Depth:", best_depth)
```

```
Depth: 3 Accuracy: 0.9771
Depth: 5 Accuracy: 0.9974
Depth: 7 Accuracy: 0.99995
Best Depth: 7
```

In [18]:
```
best_dt = DecisionTreeClassifier(max_depth=best_depth)
best_dt.fit(X_train, y_train)
```

Out[18]:

```
▾ DecisionTreeClassifier
▸ Parameters
```

In [19]:

```
y_pred = best_dt.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

```
Accuracy: 0.99995
Precision: 0.9991126885536823
Recall: 1.0
F1 Score: 0.9995561473590768
```

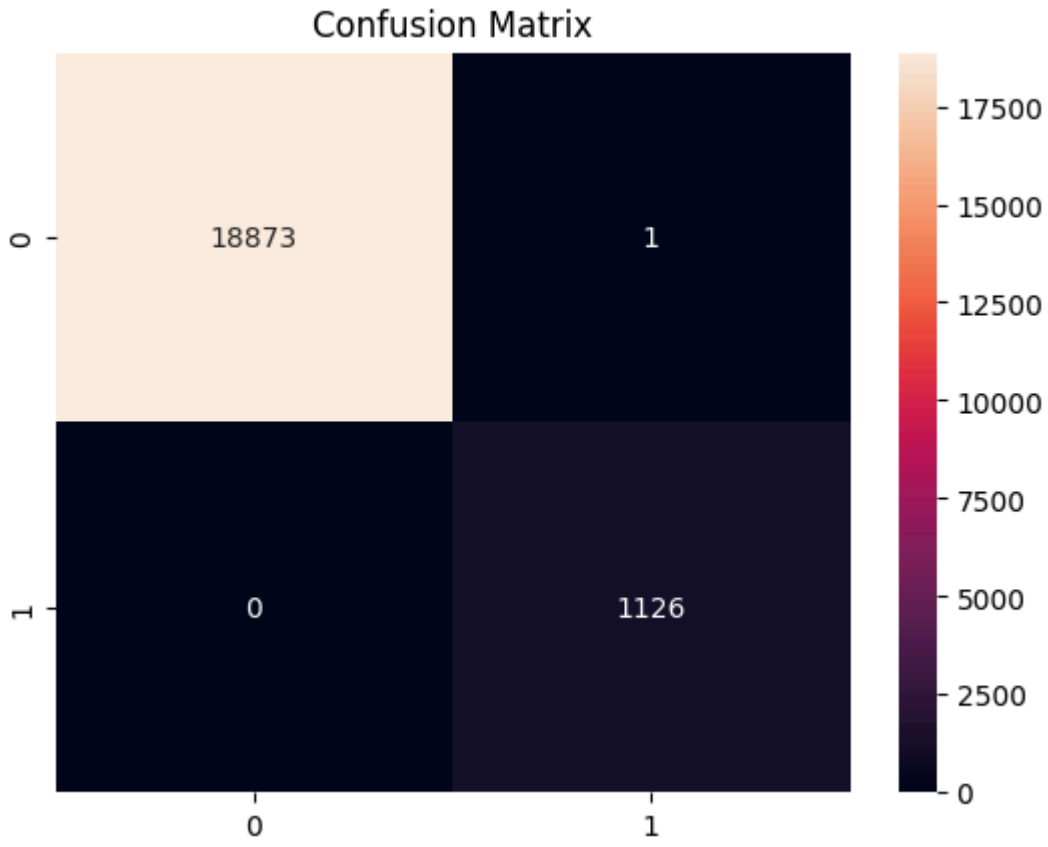
In [20]:

```
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[18873     1]
 [    0  1126]]
```

In [21]:

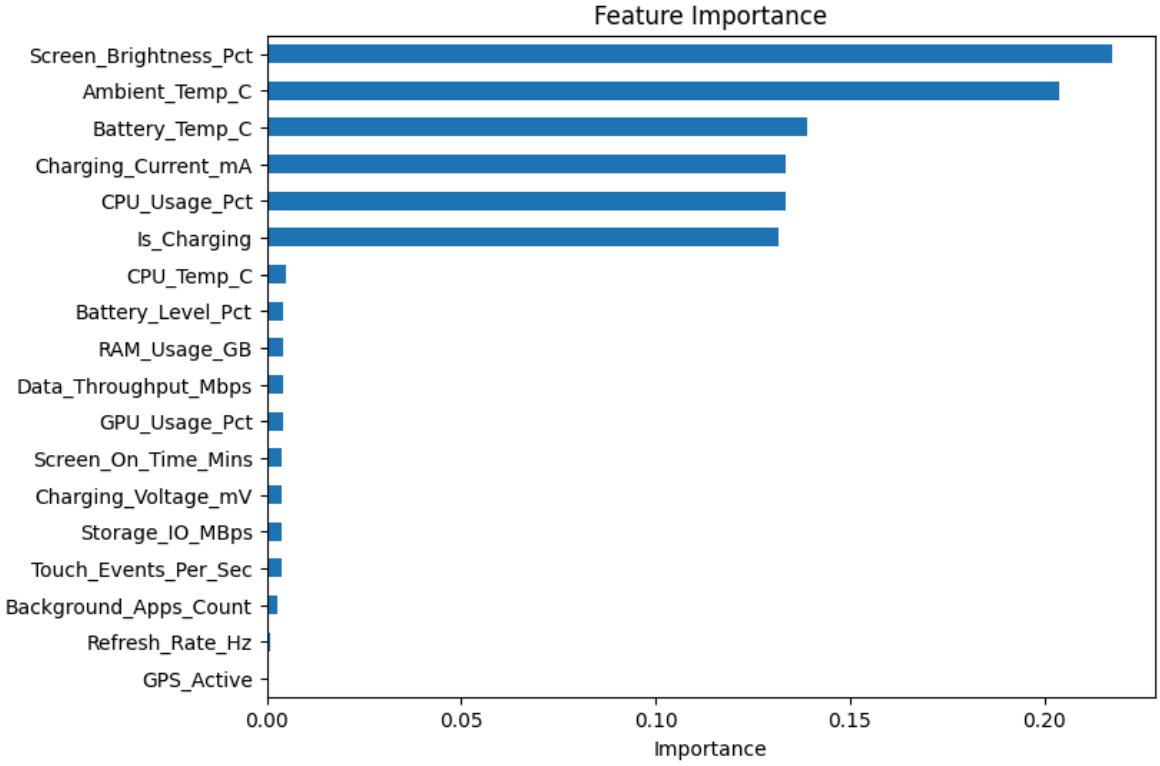
```
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.show()
```



In [23]:

```
rf.fit(X_train, y_train)

importances = rf.feature_importances_

feat_imp = pd.Series(importances, index=df.columns[:-1])

feat_imp = feat_imp.sort_values()
```

```
plt.figure(figsize=(8,6))
feat_imp.plot(kind='barh')

plt.title("Feature Importance")
plt.xlabel("Importance")
plt.show()
```



In [ ]:

In [ ]: